

EXPERIMENT – 9

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Low-Level File Operations using lseek() and File Descriptor Redirection using dup2()

Aim

To perform low-level file operations and demonstrate the use of lseek(), compare low-level file operations with standard library file operations, and perform file descriptor redirection using dup2().

Theory

- **lseek()** – Changes the current file offset of an open file descriptor.
- **Standard library file operations** – Functions such as fopen(), fgets(), fputs(), and fclose() provide higher-level file handling.
- **dup2()** – Duplicates a file descriptor and can be used to redirect standard input or output to a file.

Algorithm

1. Start the program.
2. Open input.txt using open().
3. Create output.txt using open().
4. Read data from input.txt using read().
5. Write the data to output.txt using write().
6. Use lseek() to reset the file position.
7. Close the low-level file descriptors.
8. Open the input and output files using standard library functions.
9. Copy the contents using fgets() and fputs().
10. Open redirected_output.txt.
11. Use dup2() to redirect standard output to the file.
12. Write output using printf().
13. Close the files and stop the program.

Program Code

```
k_anusri@DESKTOP-TM20BAQ:~$ cat > exp9_file_operations.c <<'EOF'
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t n;

    source = open("input.txt", O_RDONLY);

    if (source < 0)
    {
        perror("open");
        return 1;
    }

    destination = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("open");
        close(source);
        return 1;
    }

    while ((n = read(source, buffer, sizeof(buffer))) > 0)
        write(destination, buffer, n);

    lseek(source, 0, SEEK_SET);

    close(source);
    close(destination);

    FILE *in = fopen("input.txt", "r");
    FILE *out = fopen("stdlib_output.txt", "w");

    if (in == NULL || out == NULL)
    {
        perror("fopen");
        return 1;
    }

    while (fgets(buffer, sizeof(buffer), in) != NULL)
        fputs(buffer, out);

    fclose(in);
    fclose(out);

    int fd = open("redirected_output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("Output redirected using dup2().\n");
    printf("This output is written to redirected_output.txt\n");

    return 0;
}
EOF
```

Execution Screenshots

Screenshot 1 – Low-Level File Copy

```
k_anusri@DESKTOP-TM20BAQ:~$ cat output.txt
Experiment 9 file operations using lseek and dup2.
```

Screenshot 2 – Standard Library File Copy

```
k_anusri@DESKTOP-TM20BAQ:~$ cat stdlib_output.txt
Experiment 9 file operations using lseek and dup2.
```

Screenshot 3 – dup2() File Redirection

```
k_anusri@DESKTOP-TM20BAQ:~$ cat redirected_output.txt  
Output redirected using dup2().  
This output is written to redirected_output.txt
```

Result

The program successfully performed low-level file copying using `open()`, `read()`, and `write()`. The file operation was compared with standard library file handling, and output redirection was successfully demonstrated using `dup2()`.